

Function Calling • MCP • Code Mode

复杂 Agent 工具调用的结构性瓶颈与下一代编排范式

以金字塔原理组织：结论先行 • 归类分组 • 逻辑递进

金字塔总览：三者不在同一抽象层

结论：混合架构
不是三选一

Code Mode：解决复杂编排、上下文与数据流问题

MCP：解决工具连接、发现、协议与生态标准化

Function Calling：解决模型如何发出结构化、可验证的原子动作

图 1 金字塔结论：MCP 与 Function Calling / Code Mode 不在同一抽象层

研究与整理日期：2026-08-21

执行摘要：先给结论

核心结论：Function Calling 最适合“一个明确动作一次调用”的原子交互；MCP 解决工具连接与协议标准化；Code Mode 则针对复杂 Agent 工作流，把多步工具编排、数据处理和控制流从反复的 LLM 推理回合下沉到可执行代码中。生产系统最合理的答案通常不是三选一，而是 Hybrid。

一句话记忆

Function Calling = 模型发 RPC；MCP = 工具的 USB-C 协议；Code Mode = 模型临时写一个小程序来组合 RPC。

问题	传统 Function Calling 的结构瓶颈	Code Mode 的对应解法
模型调用回合	每个工具结果通常返回模型，再推理下一步	一次生成局部程序，程序内完成多个有界步骤
Context	Schema、历史和中间结果竞争同一窗口	中间数据留在 sandbox，只返回必要结果
工具规模	大量工具定义前置加载导致 schema bloat	Tool Search / progressive disclosure 按需加载接口
组合能力	工具是离散动作，循环/分支/聚合需模型逐步驱动	代码原生表达 loop / if / map / join / retry / concurrency
数据处理	模型被迫“读表、求和、过滤、拼接”	让代码做 deterministic computation
副作用治理	调用边界清晰，易审批、审计	仍需 capability gateway；高风险写操作常保留 direct call

注意：Code Mode 并不消灭 Function Calling。它把 Function Calling 从“模型每一步直接控制”降级为“代码运行时可以调用的底层 primitive”。MCP 也不等于 Function Calling：MCP 只是规定工具如何被发现、描述、连接和调用，模型可以把同一个 MCP 工具暴露成 direct call，也可以暴露在 Code Mode 中。[2][4]

正确的抽象关系：MCP 是 Tool Source / Protocol，Function Calling 与 Code Mode 是 Model Interface

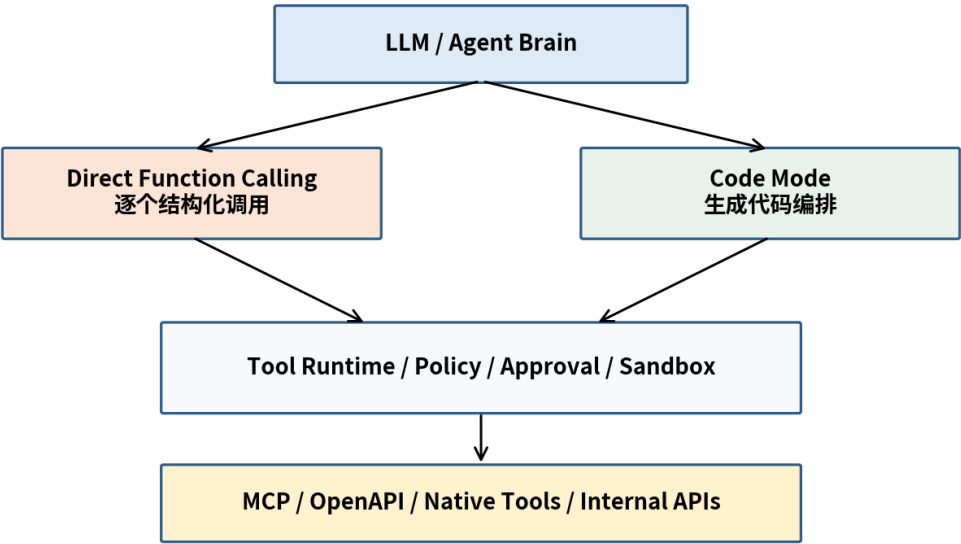


图 2 三者的正确抽象关系

目录与阅读路径

1. 三者到底解决哪一层问题：先校准抽象层。
2. Function Calling 的工作机制，以及为什么它曾经是关键突破。
3. Function Calling 的七类结构性瓶颈：它为什么在复杂 Agent 中开始“变贵、变慢、变脆”。
4. MCP 解决什么、没有解决什么，以及为什么 MCP 普及反而放大工具规模问题。
5. Code Mode / Programmatic Tool Calling 的核心机制与 know-why。
6. 逐项映射：Code Mode 如何解决 Function Calling 的瓶颈。
7. Code Mode 的新风险：sandbox、工具契约、静默错误、审批与可观测性。
8. 选型矩阵：什么时候 Direct，什么时候 Code，什么时候 Hybrid。
9. 生产实现蓝图、迁移路径与评测指标。
10. 案例推演与最终原则。

推荐阅读

如果只想抓住架构本质，重点读第 3、5、6、8、9 章；如果要做 Agent Runtime，实现层重点读第 7、9 章。

1. 先校准抽象层：Function Calling、MCP、Code Mode 不是同类东西

很多讨论把这三者放在一个“工具调用技术”列表里比较，导致概念错位。更准确的拆法是：Tool Source / Protocol、Model Interface、Execution Runtime 是三个独立维度。Cloudflare 2026 年的 Agents 文档也明确把它们拆成：模型接口（Direct Tool Calls vs Code Mode）、执行位置、工具来源（Native / MCP）。[4]

维度	核心问题	代表机制
Tool Source / Protocol	工具在哪里？如何标准化描述、发现、连接、协商能力？	MCP、OpenAPI、Native SDK
Model Interface	模型以什么动作语言表达“我要调用工具”？	Function Calling、Code Mode
Execution Runtime	谁执行？在哪里执行？如何隔离、审批、记录？	Host Runtime、Sandbox、Worker、Browser
Policy / Authority	哪些动作允许执行？谁授权？	Capability、Approval、RBAC、Audit

MCP 采用 client-host-server 架构，基于 JSON-RPC，强调 Host 管理连接、能力协商和安全边界；MCP Server 暴露 Tools / Resources / Prompts 等能力。协议本身并不强制“模型必须直接逐个调用”。[2][3]

关键认知

MCP 解决的是 integration fragmentation；Code Mode 解决的是 orchestration inefficiency。MCP 可以成为 Code Mode 的底层工具来源。

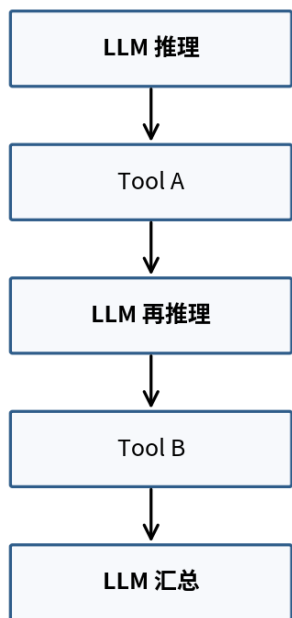
2. Function Calling：为什么它曾是一个关键突破

Function Calling 把模型输出从“自然语言里说我要调用某个 API”升级为结构化动作。模型根据工具名称、描述和 JSON Schema 选择工具并生成参数，Host 负责真正执行，然后把 tool_result 返回给模型。现代实现还可以启用严格 Schema 约束，提高参数结构可靠性。[1]

```
while True:
    response = LLM(messages, tools=tool_schemas)
    if response.type != "tool_call":
        break

    result = execute(response.name, response.arguments)
    messages.append(tool_result(result))
```

Direct Function Calling



Code Mode / Programmatic Tool Calling

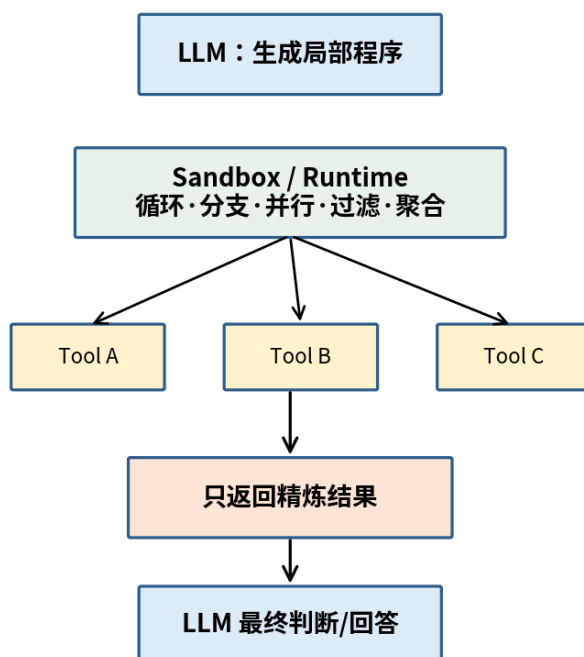


图3 Direct Function Calling 与 Code Mode 的控制流差异

Function Calling 的工程价值非常大：它提供显式动作边界、输入 Schema、统一的 Host 执行点和可观测的调用记录。这使权限检查、用户批准、参数校验、重试与审计都容易插入。对单个 API 查询、一次写操作、需要模型看完结果再重新判断的任务，它依然通常是最好的选择。

3. Function Calling 的潜在缺陷：七类结构性瓶颈

这里的“缺陷”不是说 Function Calling 错了，而是指：当工具数量、调用步数、返回数据量和数据依赖增长时，逐步 Tool Loop 的成本呈系统性放大。Anthropic 2025 年把复杂工作流中的两个根本问题概括为：中间结果污染 Context，以及每次工具调用都需要完整模型推理的 inference overhead。[5]

3.1 缺陷一：每一步都把控制权交回 LLM，形成 inference tax

Direct Tool Loop 的基本节拍是“LLM → Tool → LLM → Tool”。如果一个任务有 20 个依赖调用，极端情况下会产生接近 20 个模型决策回合。真正昂贵的不是 API 本身，而是每次都重新进行语言模型推理、读取上下文、决定下一步。

这导致两个后果：第一，latency 随步骤数累积；第二，模型在许多机械步骤上重复花费推理预算，例如分页、等待、逐行求和、判断状态码、组装数组。

本质

Function Calling 把 LLM 既当“语义决策器”，又当“工作流解释器”。复杂任务中，后者常常是不必要的。

3.2 缺陷二：Context Window 被迫兼任数据总线

工具结果通常先回到模型 Context，模型才能发下一次调用。于是 Context 同时承担两种职责：高价值 reasoning memory，以及低价值 intermediate data bus。大量日志、表格、搜索结果、API payload 即使只需要其中 1% 信息，也可能整个进入上下文。

Function Calling：Context 兼做“数据总线” **Code Mode：Context 回归“推理工作内存”**

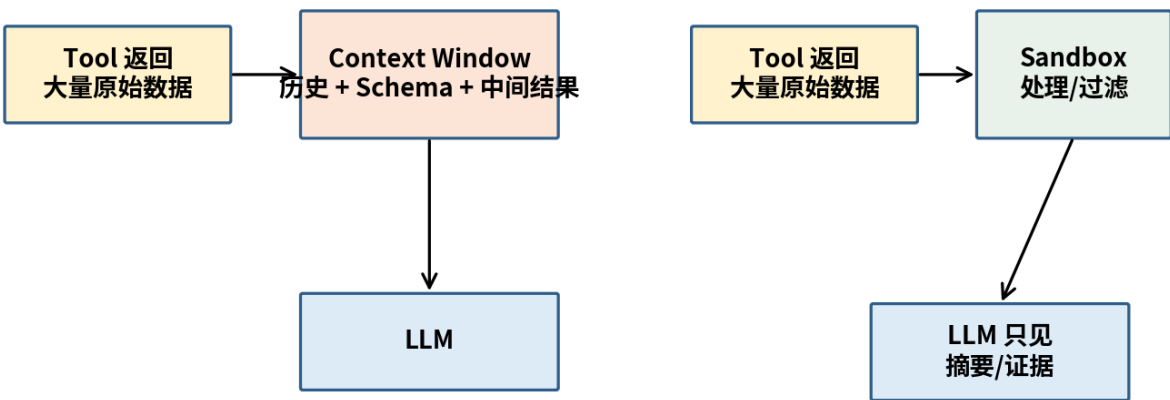


图 4 Function Calling 容易让 Context Window 兼任“数据总线”

Anthropic 在 Code Execution with MCP 中给出一个极端但非常说明问题的例子：将 MCP 工具按代码 API 按需读取，可把一个场景中工具相关上下文从约 150,000 tokens 降到约 2,000 tokens，节省 98.7%。这个数字是特定案例，不应外推为普遍比例，但它揭示了结构性浪费来自哪里。[6]

3.3 缺陷三：工具越多，Schema 本身就越像“静态税”

Function Calling 往往把所有候选工具的名称、描述和参数 Schema 预先注入模型。小规模时没有问题；连接多个 MCP Server 后，工具数量可以迅速从十几个增长到数十、数百甚至更多。

Anthropic 报告过一个五服务器示例：GitHub、Slack、Sentry、Grafana、Splunk 共 58 个工具，约占 55K tokens；再增加 Jira 等系统会逼近 100K+。他们还观察过优化前工具定义占 134K tokens 的内部案例。[5]

反直觉效应

MCP 越成功、连接越容易，Agent 越容易拥有更多工具；如果仍采用“全量 Schema + Direct Tool Calling”，MCP 的生态扩展性会转化成 Context 压力。

3.4 缺陷四：Function Call 是散作，合能力主要反推理

函数调用的原子 action space 通常是：search()、read()、write()、update()。但实际任务往往需要“先搜索 → 遍历 → 条件过滤 → 按 ID join → 并行查询 → 聚合 → 取 Top-K”。Direct Tool Calling 可以完成，但这些控制结构需要模型一次次决定。

CodeAct 论文从学术角度指出：JSON / 文本式预定义动作存在受限 action space 和难以组合多个工具的问题；将 executable Python 作为 action space，在其评测中最高取得约 20 个百分点的成功率提升。[9]

3.5 缺陷五：把 deterministic computation 错交给概率模型

过滤 10,000 条记录、按字段排序、求和、去重、做 hash join、分页、retry、构造批量请求，这些都是经典程序擅长的确定性计算。Direct Function Calling 经常让 LLM “看一眼数据然后手工综合”，既浪费 token，又增加漏项、算错、重复的概率。

工程原则

不要用模型做 CPU 应该做的事情。LLM 负责语义判断；代码负责机械计算。

3.6 缺陷六：复杂工具集中的 selection / parameter ambiguity

工具增多时，名字相近、参数复杂、返回结构相似，会放大 wrong tool、wrong arguments、参数组合不一致等错误。严格 JSON Schema 可以保证“结构合法”，但不能完全表达“何时应填这个可选字段”“哪个 ID 格式属于哪个系统”“两个参数之间的业务约束”。Anthropic 的 Tool Use Examples 正是为补足这类语义契约而提出。[5]

3.7 缺陷七：跨调用状态、并发与错误处理表达笨重

程序语言天然具备局部变量、循环、异常、并发、缓存、函数封装。Direct Tool Loop 当然也能通过 Agent State 实现，但每个结构都要在 Host 或模型推理循环中额外编码。随着 workflow 复杂度提升，它逐渐接近“用对话协议手写一个 workflow engine”。

4. MCP：它解决了 Function Calling 的哪部分问题？

MCP 的核心价值是标准化“AI 应用 ↔ 外部系统”之间的接口。Server 暴露能力，Client / Host 建立连接、协商 capability，并通过统一协议发现与调用。它解决的是过去每个 Agent 框架都要为每个 SaaS / DB / 文件系统写一套专有 integration 的碎片化问题。[2][3]

问题	MCP 是否解决	说明
跨系统集成碎片化	是	统一工具/资源/提示等协议与发现机制
模型如何选择工具	部分	提供元数据，但最终选择仍是模型/Host 的 orchestration
每次调用都回 LLM	否	协议不规定模型接口必须 direct
中间结果污染 Context	否	由 Agent Runtime / Model Interface 决定
大工具集 Schema bloat	不自动解决	MCP 让工具更多；需要 Tool Search / deferred loading / Code Mode
循环、分支、聚合	否	这是 orchestration / runtime 层问题
安全边界	提供基础原则	Host 管理连接与权限；高风险动作仍需实际策略与审批

因此，MCP 与 Function Calling 的关系更像：MCP Server 提供工具目录和调用协议，Host 可以把它们转成模型看得懂的 Function Tools；也可以把它们映射成 Code Mode 的 typed methods。Cloudflare 2026 文档明确支持这两种模式。[4][8]

4.1 为什么 MCP 的成功反而催生 Code Mode

当 MCP 让连接 5 个系统变得容易以后，系统很快会变成 20 个服务器、几百个工具。于是早期“所有工具 Schema 都塞给模型”的 Function Calling 设计出现规模瓶颈。这不是 MCP 的失败，而是“连接层扩展速度超过了模型接口层”的结果。

演进逻辑

Function Calling 解决“模型终于能结构化调用一个工具” → MCP 解决“工具终于能标准化接入” → Code Mode 解决“工具太多、调用太复杂以后如何高效编排”。

5. Code Mode：它到底改变了什么？

Code Mode 的核心不是“把 JSON 换成 JavaScript/Python”。真正的架构变化是：LLM 不再逐步解释 workflow，而是先生成一个局部可执行程序（local controller），由 sandbox 运行这个程序并调用底层工具。只有程序最终选择的结果进入模型 Context。Cloudflare 将其定义为：模型接收一个 code-execution tool，代码本身成为紧凑计划，调用工具、处理结果并返回所需信息。[7]

```
# Direct Function Calling
LLM -> get_team_members() -> LLM
LLM -> get_expenses(A) -> LLM
LLM -> get_expenses(B) -> LLM
...
LLM -> sum / compare / answer

# Code Mode
LLM -> generate program
program -> get_team_members()
program -> parallel get_expenses(*)
program -> sum / filter / compare
program -> return only exceeded_members
LLM -> final answer
```

5.1 Know-why 1：把 Context 从 Data Plane 解耦

Code Mode 允许工具 A 的大结果直接在执行环境中流向工具 B 或本地过滤器，而不必先完整进入模型。于是形成更清晰的分层：LLM 是 Control / Semantic Plane；sandbox 是 Data / Mechanical Plane。Anthropic 明确指出，Programmatic Tool Calling 可以让中间结果被代码处理，而不是被模型消费。[5]

5.2 Know-why 2：把 repeated inference 成 deterministic execution

一次模型推理生成一段程序后，for/if/try/async/gather 等结构可以执行很多步。可以把它理解为“局部策略编译”：模型先生成 task-specific controller，然后 controller 在边界内执行。这样将昂贵的逐步 inference 摊销成一次规划 + 多次廉价的确定性执行。

5.3 Know-why 3：把离散 Tool Actions 提升可合 Program

Function Calling 给模型的是有限 primitive；Code Mode 让 primitive 通过程序结构组合。组合能力不再只来自模型“记住前一步并继续调用”，而是来自编程语言本身：局部状态、循环、函数、分支、并发、错误处理和数据结构。

5.4 Know-why 4：Progressive Disclosure，让大型工具目录按需进入上下文

Code Mode 可以结合 filesystem-style discovery、search_tools 或 codemode.search()/describe()，只加载当前任务需要的类型。Cloudflare 2026 Code Mode 文档把 progressive discovery 作为大型 tool catalog 的核心机制；Anthropic 的 Tool Search 也采用 defer loading。[5][7]

6. 逐项映射：Code Mode 解决了 Function Calling 的什么问题？

Function Calling 瓶颈	根因	Code Mode 机制	效果
每步 inference	控制流由模型逐步解释	一次生成局部程序	减少模型往返与决策回合
中间结果堆 Context	结果必须给模型才能继续	结果留在 sandbox	减少 token、保留推理空间
工具定义膨胀	所有候选接口预加载	search/describe / deferred loading	按需加载类型与文档
多工具组合笨重	Action Space 是原子调用	代码表达组合	loop/branch/join/aggregate 更自然
大量机械计算	LLM 手工综合	程序内 filter/sort/map/reduce	更确定、更便宜、更可测试
并发难表达	模型逐个发调用	Promise.all / asyncio 等	降低 wall-clock latency
重复工作流	每次重新推理	可保存/复用 snippet	将成功轨迹固化为程序资产

6.1 数据规模：从“让模型读原始数据”变成“让模型读证据”

Anthropic 在 Programmatic Tool Calling 示例中描述：大量 expense line items 可以由代码求和、比较，模型最终只看到超预算人员，而不是 2,000+ 原始条目。[5] 这并不是简单的摘要压缩，而是把“选择什么进入模型”变成程序化 context engineering。

6.2 工具规模：从“加载全部 API 文档”变成“运行时发现”

Anthropic 的 Tool Search 示例中，50+ MCP tools 原本约 72K tool tokens；改为搜索工具约 500 tokens + 只展开 3-5 个相关工具后，总体上下文显著下降，并报告约 85% token reduction 的内部测试结果。[5]

6.3 经验数据：Code Mode 的收益不是只停留在理论

来源	观测	正确解读
Anthropic Code Execution with MCP	特定示例：150K → 2K tool-related tokens (98.7%)	说明 progressive disclosure + code dataflow 的上限潜力，不代表所有任务
Anthropic Advanced Tool Use	大型工具库 Tool Search 内测约 85% token reduction；部分 MCP eval accuracy 提升	大 catalog 时 schema management 本身会影响准确率
Cloudflare	把 MCP server 转成 TypeScript API 的实验曾报告约 81% token reduction	代码接口可压低 schema + round-trip 成本
CodeAct, ICML 2024	17 个模型上，Executable Code Actions 最高约 +20 个百分点 success rate	说明 code-as-action 在复杂组合任务中具有表达优势

重要：这些数字来自不同产品、任务与评测，不能直接横向相加。它们共同支持的是“复杂、多工具、大数据工作流中，代码式编排具有结构性效率优势”这一方向。[5][6][9][10]

7. Code Mode 不是免费午餐：它引入了新的复杂性

Code Mode 解决了 Function Calling 的规模问题，却把系统带入“执行不可信模型生成代码”的世界。这意味着 Agent Runtime 必须更像一个真正的操作系统/ workflow 平台，而不再只是一个工具 dispatch dictionary。

7.1 新一：必有正的 Sandbox

模型生成代码应视为 untrusted code。生产实现需要隔离文件系统、网络、进程、CPU、内存、执行时限和凭证。Cloudflare 的 Code Mode 通过 isolated Worker / sandbox 执行，并强调 generated code 只能通过被提供的工具能力访问外部系统。[8][10]

安全原则

Sandbox 不应该直接持有 authority。Authority 应通过显式 capability 注入：代码能调用什么，由 Host / Gateway 决定，而不是由代码自己拿 API Key。

7.2 新风险二：Tool Contract 必须比 Function Calling 时代更完整

Function Calling 时代，输入 Schema 很重要；Code Mode 时代，模型要写“解析返回值的代码”，因此输出 shape、错误类型、分页行为、nullability、幂等性、rate limit 都必须清楚。否则代码可以语法正确却按错误字段解析，产生 silent failure。Anthropic 也明确建议为 programmatic tool calling 清晰记录 return formats。[5]

7.3 新风险三：静默逻辑错误比 JSON 格式错误更隐蔽

Code Mode 的错误不再只是 invalid JSON。可能出现错误 join key、错误数据 provenance、吞异常、错误排序方向、重试风暴、并发过量、部分成功被当成全成功。2026 年一些后续研究也开始专门研究 code-mode 中“代码执行成功但工具间契约错了”的 silent failures。这里意味着 eval 必须从 syntax validity 升级到 end-to-end task correctness。

7.4 新风险四：审批和副作用需要“可暂停、可恢复”的运行

Direct Function Calling 的每一次写操作天然经过 Host，审批很直接。Code Mode 中，一个程序可能在循环中准备多个写操作，因此 runtime 需要能在某个 connector method 前暂停、请求批准、批准后 replay 已完成步骤并恢复。Cloudflare Durable Code Mode Runtime 已提供类似 pause / approval / resume 机制。[8]

7.5 为什么高风险写操作常应保留 Direct Function Calling

OpenAI 当前 Programmatic Tool Calling 指南明确建议：如果每一步结果都会改变模型的下一步语义判断、操作需要审批、或者输出必须保留原生 citation / artifact，则倾向 direct calls；PTC 更适合 filtering、joining、ranking、deduplication、aggregation、validation 等有界处理。[11]

8. 最完整对比：Function Calling vs MCP vs Code Mode

维度	Function Calling	MCP	Code Mode
定位	模型动作接口	连接/协议标准	模型动作与编排接口
模型输出	结构化 tool call	协议不规定模型输出形式	可执行程序
适合任务	单步/少步、语义自适应	跨系统接入与生态复用	多步有界编排、大数据处理
控制流	模型逐步控制	Host/Client 决定	程序局部控制
Context 中间结果	通常进入模型	取决于上层实现	可留在 sandbox
工具目录	通常预先注入	支持发现，但不自动解决模型上下文	可 progressive discovery
数据处理	LLM 或 Host 手工逻辑	不负责	代码原生处理
审批/审计	边界清晰，最直接	Host 应提供人类控制	需要 durable pause/approval 或 gateway
安全核心	参数校验 + Host policy	连接边界 + capability negotiation	sandbox + capability + policy
主要风险	round-trip/context/schema bloat	server trust / tool explosion / integration security	代码逻辑错误、sandbox、复杂运行时

8.1 什么时候优先 Direct Function Calling

- 一个调用就能完成，或只有少量步骤。
- 每个中间结果都可能触发新的语义判断或改变研究方向。
- 动作具有明显副作用，需要逐次用户批准或审计。
- 结果需要原生 citation、artifact、UI 对象，而不应被程序压平。
- 工具返回很小，使用 Code Mode 的 sandbox 启动成本不划算。

8.2 什么时候优先 Code Mode

- 需要 3+ 个有明确数据依赖、但不需要每一步重新语义推理的调用。
- 需要并行查询几十个对象、分页、批处理。
- 需要 filter / sort / join / groupby / dedup / aggregate / validation。
- 中间结果大，但最终只需要小型结构化结论。
- 工具目录很大，需要按需 search / describe。
- 同类 workflow 反复发生，值得把成功程序保存为 snippet / skill。

8.3 什么时候必须 Hybrid

现实生产任务常由“语义节点 + 机械节点 + 副作用节点”混合组成。最优方式是：LLM 对语义节点直接判断；Code Mode 承担机械数据流；真正改变外部世界的动作穿过统一 Policy/Approval Gateway。

推荐生产架构：语义判断留给模型，机械编排交给代码，副作用交给策略层

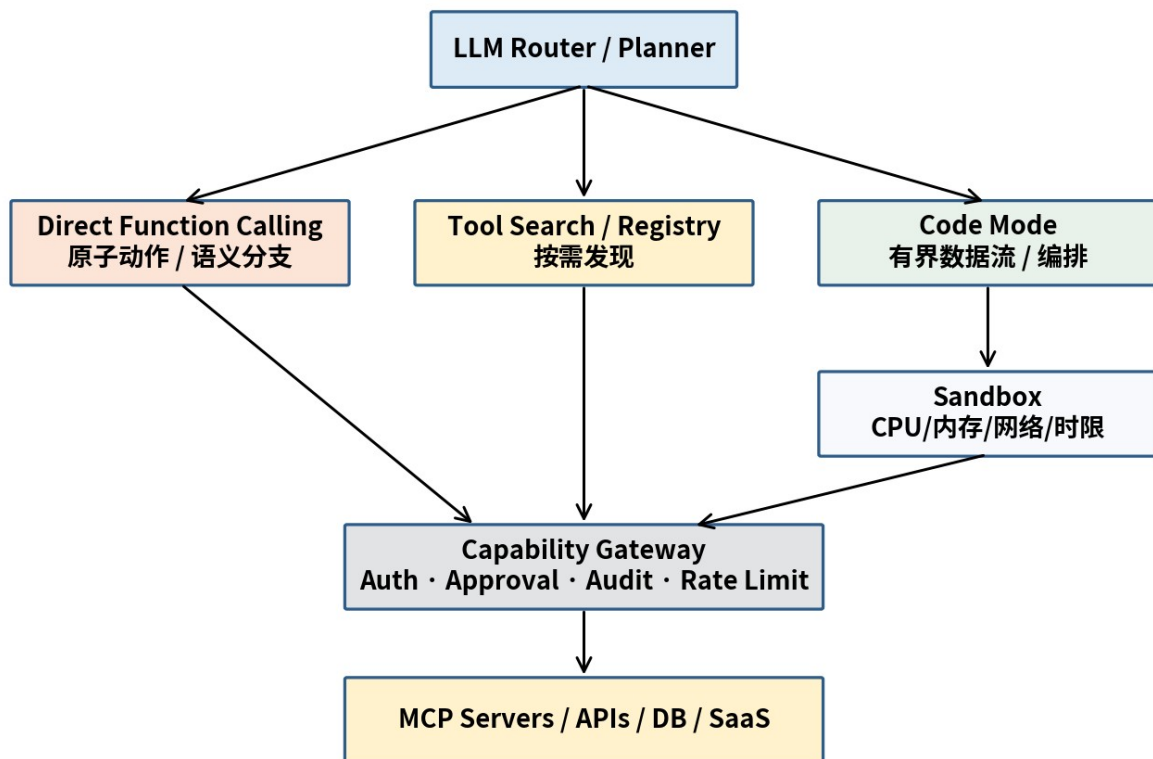


图 5 推荐的 Hybrid Agent Tool Runtime

9. 生产落地蓝图：如何从 Function Calling 演进到 Hybrid

不要一开始就把所有工具改成 Code Mode。最稳妥的迁移方式是“保留 direct baseline → 找瓶颈 → 只下沉有界阶段 → 通过 eval 验证”。

9.1 Step 1：给现有 Tool Loop 做画像

- 每任务平均 tool calls / model turns。
- tool schema tokens 占系统上下文比例。
- 中间 tool_result token 大小及其中最终被使用的比例。
- 每类调用 latency：模型推理 vs API。
- 错误类型：wrong tool、wrong args、data processing、semantic judgment、side effect。

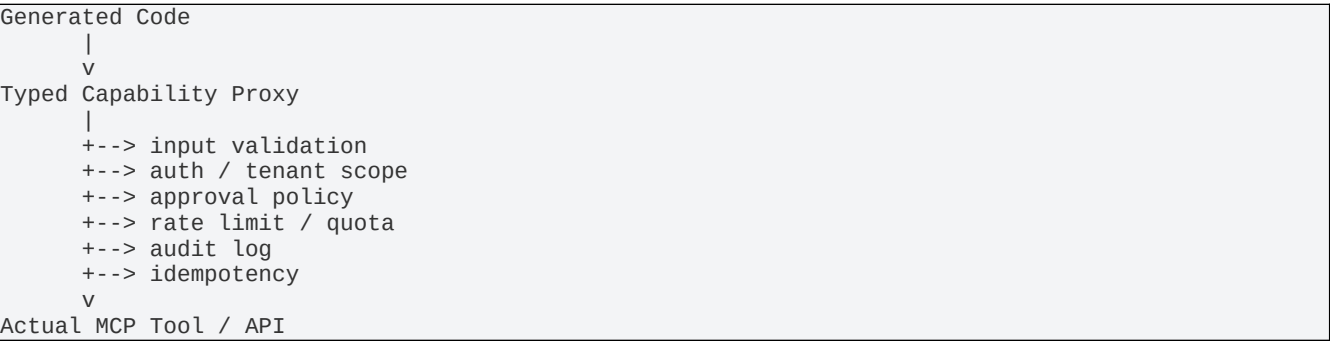
9.2 Step 2：识别“有界机械子图”

最适合下沉 Code Mode 的不是“整个 Agent”，而是工作流中的 bounded subgraph。例如：搜索 50 个仓库 → 筛 active → 查 PR → 取 blocked → 返回 10 条证据。这个子图的输入、工具集合、终止条件和输出 Schema 都可明确。

9.3 Step 3：为 Programmatic Tools 补全 Contract

Contract 项	要求
Inputs	类型、必填、枚举、ID 格式、时间格式
Outputs	字段名、类型、嵌套结构、可能为空的字段
Errors	可重试 vs 不可重试、错误码、部分成功
Side effects	是否写入、是否幂等、是否需要审批
Limits	分页、rate limit、最大 page size、并发限制
Evidence	最终回答必须保留哪些 ID / source / citation metadata

9.4 Step 4：建立 Capability Gateway，而不是把权限写进生成代码



9.5 Step 5：为 Code Runtime 设硬边界

- 最大执行时间、最大 tool-call 次数、最大循环次数。

- 并发上限与每工具 rate limit。
- 默认禁止任意网络；只允许 typed capability。
- 默认不暴露长期凭证与宿主文件系统。
- 程序 stdout / return 设大小限制，防止再次污染 Context。
- 对 transient error 限定 retry 次数，禁止无限重试。

9.6 Step 6 : 明确 Direct 与 Programmatic 的路由规则

```
if task.requires_user_approval:
    route = DIRECT_TOOL_CALL
elif each_intermediate_result_requires_semantic_judgment:
    route = DIRECT_TOOL_CALL
elif task.is_bounded and task.needs_filter_join_aggregate:
    route = CODE_MODE
elif task.has_large_tool_catalog:
    route = TOOL_SEARCH_THEN_DIRECT_OR_CODE
else:
    route = DIRECT_TOOL_CALL
```

10. 评测：不要只看“少了多少 Tool Calls”

OpenAI 当前 PTC 指南特别提醒：更少 calls / turns / intermediate outputs 只有在最终质量不下降时才算改进。
最合理的评测是 direct baseline 与 Code Mode 在同一代表性任务集上 A/B。[11]

指标族	核心指标	为什么重要
Correctness	Task success、字段完整、证据一致	Code 可能运行成功但语义错误
Tool reliability	Wrong tool、wrong args、contract violation	区分接口选择问题与程序逻辑问题
Efficiency	Input/output tokens、model turns、tool calls、latency、cost	验证 Code Mode 是否真正摊销推理
Context health	Schema tokens、intermediate tokens、compression ratio	测量 “Context 是否仍是数据总线”
Safety	未经批准写操作、权限越界、secret exposure	Code Mode 引入执行层风险
Recovery	retry success、partial failure、resume correctness	复杂 workflow 必须可恢复
Observability	trace completeness、program/tool lineage	问题要能定位到具体 program step

核心评测原则

不要把“模型少看了数据”自动当成好事。最终答案需要的 evidence 必须被显式保留下来；否则 token 降了，可信度也一起降了。

11. 案例推演：从 Direct Tool Loop 重构为 Code Mode

任务：找出 Engineering 团队 Q3 差旅超预算的成员，并给出姓名、实际支出、预算、超出金额。

11.1 Direct Function Calling

```
1. LLM -> get_team_members("engineering")
2. result(20 members) -> LLM
3. LLM -> get_expenses(member_1, "Q3")
4. result(50-100 lines) -> LLM
5. ... repeat 20 times ...
6. LLM -> get_budget_by_level(...)
7. LLM manually aggregates / compares
8. answer
```

问题不是它做不到，而是：20 人 × 50-100 行 expense 都可能进入 Context；每一批数据都要让模型重新参与；求和/比较属于机械计算。

11.2 Code Mode

```
team = get_team_members("engineering")

expenses = parallel_map(
    team,
    lambda m: get_expenses(m.id, "Q3")
)

result = []
for member, rows in zip(team, expenses):
    spent = sum(x.amount for x in rows)
    budget = get_budget_by_level(member.level)
    if spent > budget:
        result.append({
            "name": member.name,
            "spent": spent,
            "budget": budget,
            "over": spent - budget
        })

return sorted(result, key=lambda x: x["over"], reverse=True)
```

模型只需要看到最终超预算列表。这恰好对应 Anthropic 对 Programmatic Tool Calling 的预算合规示例。[5]

11.3 Hybrid：如果最后一步要自动通知经理

推荐把“计算谁超预算”留在 Code Mode；而“给经理发送消息”作为 side effect，由 Direct Tool Call 或 capability gateway 逐项审批。这样既保留 Code Mode 的数据效率，又保留 Function Calling 的显式授权边界。

12. 最终心智模型：把 Agent 看成编译器 + Runtime

复杂 Agent 不应该把整个世界都塞进一次 Context，再让模型逐步“口头解释执行”。更强的架构是：模型负责把自然语言意图编译成一个有限计划；Runtime 负责执行、隔离、检查、记录；必要时再把少量高价值 observation 交回模型。

层	职责	不该承担的事情
LLM / Planner	意图理解、语义判断、策略选择	大规模过滤、排序、分页、重试
Tool Search	发现相关能力	执行业务逻辑
Code Runtime	有界控制流、状态、数据处理	持有无限权限或裸凭证
Capability Gateway	授权、审批、审计、限流	语义推理
MCP / APIs	提供标准能力	决定 Agent 全局策略

最终原则

LLM decides. Code computes. Tools act. Sandbox contains. Policy authorizes. Context carries only what reasoning needs.

12.1 一张选型表直接落地

任务形态	首选
单工具查询 / 小结果	Direct Function Calling
少量调用，但每步都要重新理解结果	Direct Function Calling
批量读取 + filter/sort/join/aggregate	Code Mode
几十个并行 lookup / pagination	Code Mode
大工具目录	Tool Search / Progressive Disclosure
MCP + 大目录 + 多步数据流	Tool Search + Code Mode
高风险写操作 / 用户批准	Direct + Approval / Gateway
语义研究 + 机械数据处理混合	Hybrid

13. 用金字塔原理重新压缩全文

顶层结论：Code Mode 不是用代码取代 Function Calling，而是把“复杂、多步、数据密集型工具编排”从模型逐步推理中抽离出来；MCP 提供统一连接层；Function Calling 仍作为原子动作与审批边界存在。

1. 第一层理由：Function Calling 的瓶颈主要来自 repeated inference、context pollution、schema bloat、有限组合能力和机械计算错配。
2. 第二层理由：MCP 只解决连接与标准化，不解决模型编排，因此工具生态扩张后必须引入 discovery 与新的执行接口。
3. 第三层理由：Code Mode 用程序承载控制流和数据流，允许中间结果留在 sandbox，并通过 progressive disclosure 控制工具定义成本。

4. 第四层约束：Code Mode 增加执行风险与 runtime 复杂度，因此必须配合 sandbox、capability、approval、contract 和 eval。
5. 最终工程答案：Direct Function Calling + Tool Search + Code Mode + Policy Gateway 的 Hybrid Runtime。

参考资料与证据来源

[1] OpenAI API Reference / Function tools

Function tools 使用 JSON Schema；strict 参数可用于严格参数校验。

<https://platform.openai.com/docs/api-reference/responses-streaming/response/refusal?lang=python>

[2] Model Context Protocol - Architecture

MCP client-host-server 架构、能力协商、安全边界与可组合性。

<https://modelcontextprotocol.io/specification/2025-06-18/architecture>

[3] Model Context Protocol - Tools

MCP Server 如何暴露可被模型调用的 tools；协议不强制具体用户交互方式。

<https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

[4] Cloudflare Agents - Tools Concepts

把 Model Interface、Execution Location、Tool Source 分成三个独立设计维度。

<https://developers.cloudflare.com/agents/concepts/tools/>

[5] Anthropic - Introducing advanced tool use on the Claude Developer Platform

Tool Search、Programmatic Tool Calling、工具规模与内部实验数据。

<https://www.anthropic.com/engineering/advanced-tool-use>

[6] Anthropic - Code execution with MCP: Building more efficient agents

工具定义与中间结果的 token 开销；code API、progressive disclosure、150K→2K 特定示例。

<https://www.anthropic.com/engineering/code-execution-with-mcp>

[7] Cloudflare Agents - Code Mode

Code Mode 定义、code as a plan、progressive discovery、direct vs code mode 使用条件。

<https://developers.cloudflare.com/agents/tools/codemode/>

[8] Cloudflare Agents - Use MCP tools with Code Mode / Code Mode MCP patterns

MCP tool 映射为 typed sandbox methods、审批与 durable runtime。

<https://developers.cloudflare.com/agents/tools/codemode/mcp/>

[9] Wang et al., Executable Code Actions Elicit Better LLM Agents, ICML 2024

CodeAct：可执行代码 action space；在 17 个模型评测中最高约 +20 个百分点成功率。

<https://proceedings.mlr.press/v235/wang24h.html>

[10] Cloudflare - Sandboxing AI agents, 100x faster

Code Mode token 节省案例与模型生成代码的 sandbox 安全需求。

<https://blog.cloudflare.com/dynamic-workers/>

[11] OpenAI - Model guidance / Programmatic Tool Calling

PTC 适合 bounded workflows；direct call 适合需 fresh model judgment、approval、citation/artifact 的情况。

<https://developers.openai.com/api/docs/guides/latest-model>